JECRC™
UNIVERSITY
BUILD YOUR WORLD

Department of Engineering

Session 2026-27

PROMPT ENGINEERING

(Course Code: BCO610A)

HW-08: RED BEFORE GREEN

Debugging & Testing with AI

Submitted By	**Submitted To**
Name – Aditya Ranjan	Mr. Ajay Sharma
Enrollment No. – 25BCON1294	Prompt Engineering
Branch – B.Tech. CSE (AI&DS)	
Semester – 3	

1. HW-08 Objective

HW-08 focuses on specification-driven debugging: prove a bug fails, fix it, and prove the same behaviour works afterward. It also compares tests written from the specification with tests generated from broken code.

Requirement	What to submit
Specification	6+ behaviour rows for the Python program.
Bug seeding	Three bug classes: arithmetic, boundary, unreachable branch.
RED stage	Specification tests fail against broken code.
GREEN stage	The same tests pass after the implementation fix.
AI test experiment	Generate tests from broken code and compare both suites.
README	Testing section updated and audited.
Git	RED commit before GREEN commit; work in session folder.

2. Program Used

Student Marks Utility with three functions: calculate_average(), is_passing(), and get_grade().

Repository files:

- session/student_utils.py
- session/test_student_utils.py

3. Specification — Behaviour Contract

Function	Must return	Purpose
calculate_average([80,90,100])	90.0	Ordinary arithmetic
calculate_average([40])	40.0	Single element; no crash
is_passing(41)	True	Above pass mark
is_passing(40)	True	Exactly at pass mark
is_passing(39)	False	Below pass mark
get_grade(95)	Distinction	Top band
get_grade(60)	First	First band
get_grade(45)	Second	Second band
get_grade(30)	Fail	Below all bands

4. Buggy Copy — Three Bug Classes

```
PASS_MARK = 40

def calculate_average(marks):
    total = 0
    for mark in marks:
        total = total + mark
    return total / (len(marks) - 1)

def is_passing(mark):
    if mark > PASS_MARK:
        return True
    return False

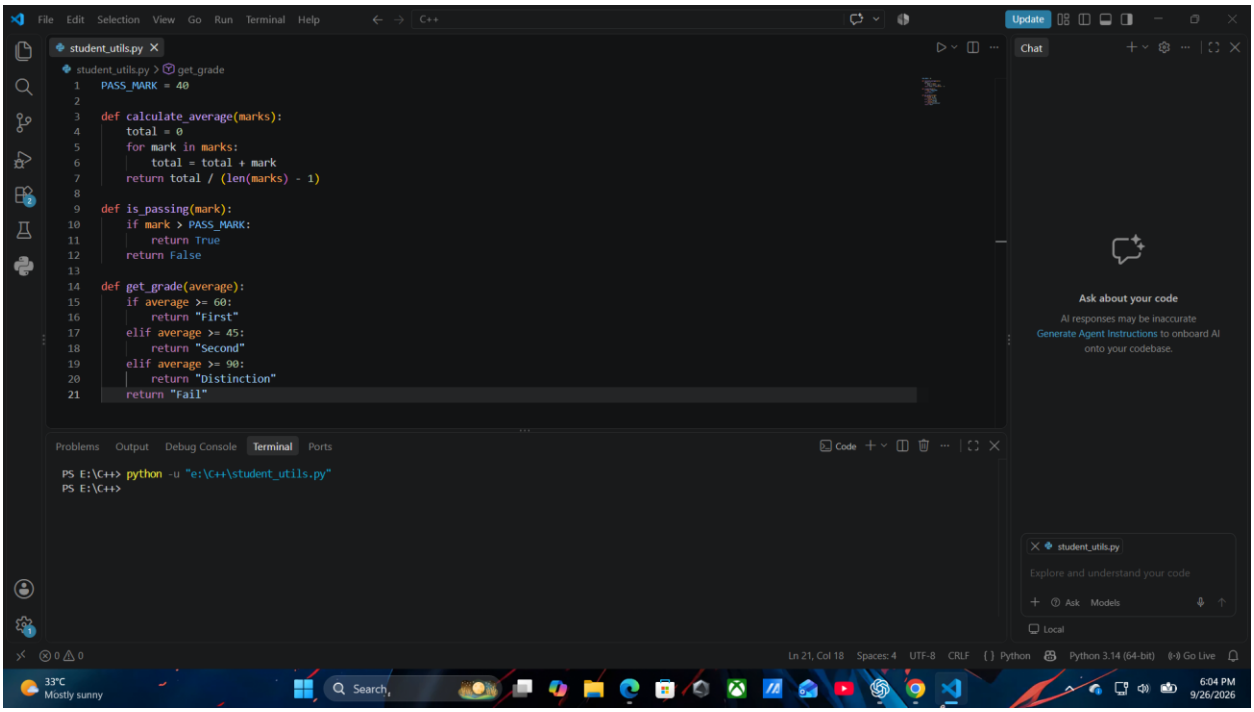
def get_grade(average):
    if average >= 60:
        return "First"
    elif average >= 45:
        return "Second"
    elif average >= 90:
        return "Distinction"
    return "Fail"
```

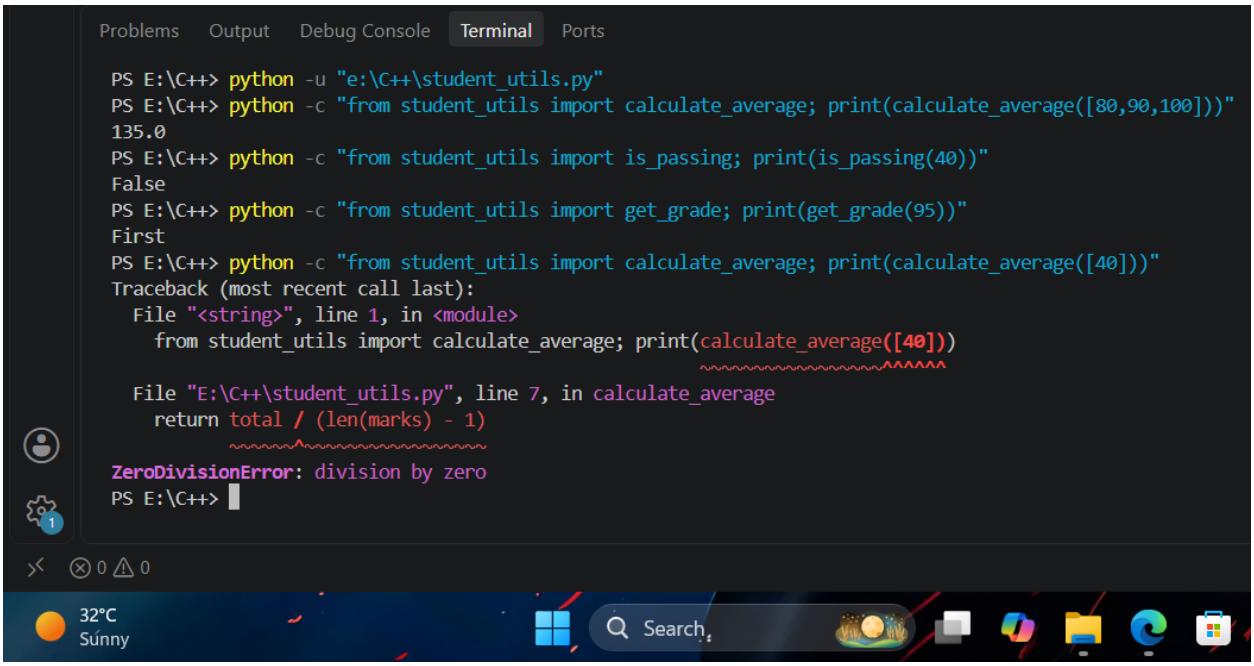
Bug class	Bug	Effect
Arithmetic	Divide by len(marks)-1	Wrong average; one mark can crash.
Boundary	Use > instead of >=	40 incorrectly fails.
Unreachable branch	>=90 check after >=60	Distinction cannot be reached.

5. Activity A — Find and Reproduce the Bugs

Run each function and reproduce every reported bug before fixing anything.

Bug	Input	Expected	Buggy result
Arithmetic	calculate_average([80,90,100])	90.0	135.0
Arithmetic	calculate_average([40])	40.0	ZeroDivisionError
Boundary	is_passing(40)	True	False
Unreachable branch	get_grade(95)	Distinction	First


File Edit Selection View Go Run Terminal Help
C++
Update
student_utils.py
student_utils.py > get_grade
PASS_MARK = 40
def calculate_average(marks):
total = 0
for mark in marks:
total = total + mark
return total / (len(marks) - 1)
def is_passing(mark):
if mark > PASS_MARK:
return True
return False
def get_grade(average):
if average >= 60:
return "First"
elif average >= 45:
return "Second"
elif average >= 90:
return "Distinction"
return "Fail"
Chat
Ask about your code
AI responses may be inaccurate
Generate Agent Instructions to onboard AI onto your codebase.
Problems Output Debug Console Terminal Ports
Code
PS E:\C++> python -u "e:\C++\student_utils.py"
PS E:\C++>
student_utils.py
Explore and understand your code
Ask Models
Local
Ln 21, Col 18 Spaces: 4 UTF-8 CRLF { } Python Python 3.14 (64-bit) Go Live
33°C
Mostly sunny
Search
6:04 PM
9/26/2026


Problems Output Debug Console Terminal Ports
PS E:\C++> python -u "e:\C++\student_utils.py"
PS E:\C++> python -c "from student_utils import calculate_average; print(calculate_average([80,90,100]))"
135.0
PS E:\C++> python -c "from student_utils import is_passing; print(is_passing(40))"
False
PS E:\C++> python -c "from student_utils import get_grade; print(get_grade(95))"
First
PS E:\C++> python -c "from student_utils import calculate_average; print(calculate_average([40]))"
Traceback (most recent call last):
File "<string>", line 1, in <module>
from student_utils import calculate_average; print(calculate_average([40]))
File "E:\C++\student_utils.py", line 7, in calculate_average
return total / (len(marks) - 1)
ZeroDivisionError: division by zero
PS E:\C++>
0 0
32°C
Sunny
Search

6. Activity B — Specification-Based Tests

```
import unittest
from student_utils import calculate_average, is_passing, get_grade

class TestStudentUtils(unittest.TestCase):

    def test_average_of_three_marks(self):
        self.assertEqual(calculate_average([80, 90, 100]), 90.0)

    def test_average_of_one_mark(self):
        self.assertEqual(calculate_average([40]), 40.0)

    def test_passing_above_mark(self):
        self.assertEqual(is_passing(41), True)

    def test_passing_at_mark(self):
        self.assertEqual(is_passing(40), True)

    def test_passing_below_mark(self):
        self.assertEqual(is_passing(39), False)

    def test_distinction_grade(self):
        self.assertEqual(get_grade(95), "Distinction")

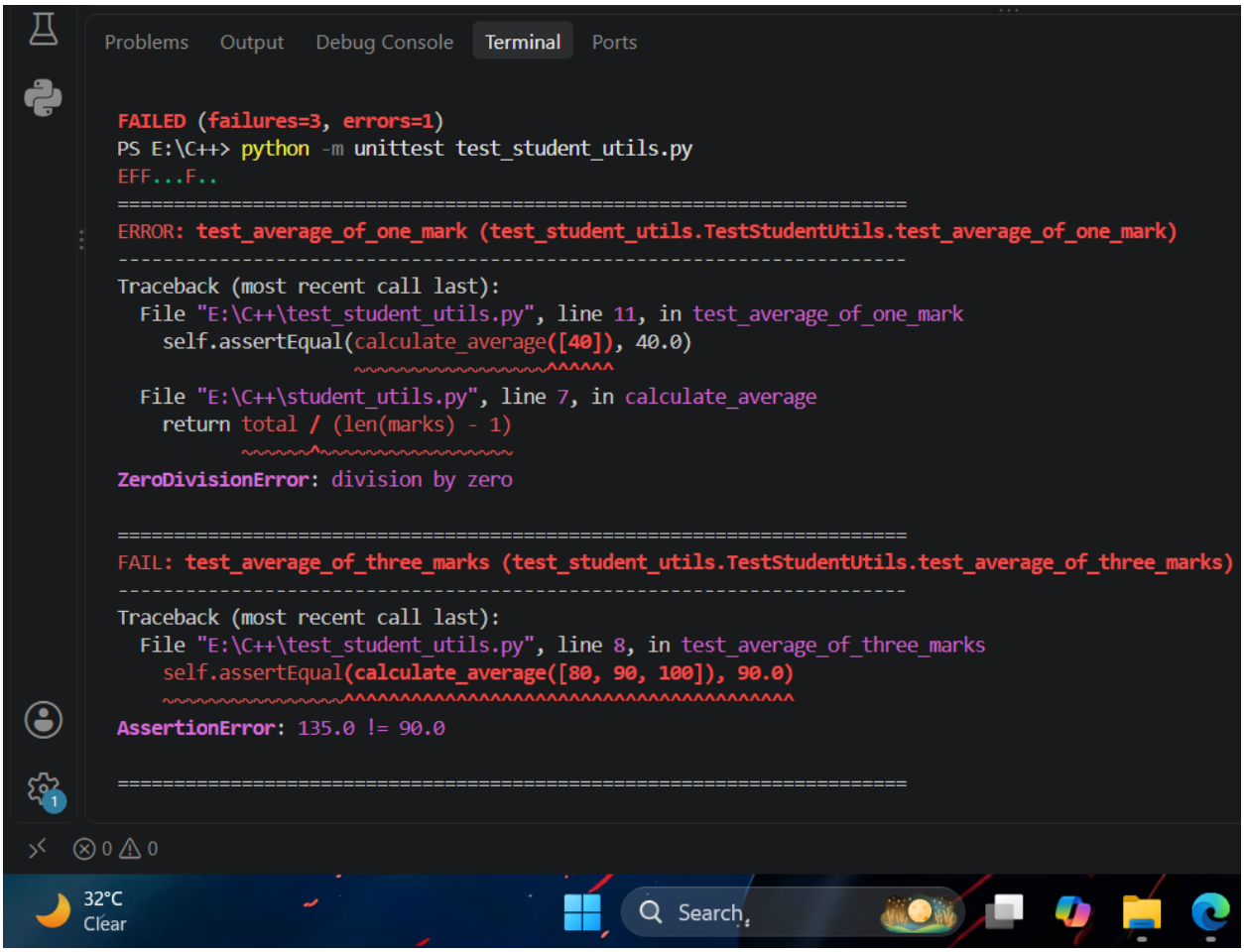
    def test_first_grade(self):
        self.assertEqual(get_grade(60), "First")

    def test_second_grade(self):
        self.assertEqual(get_grade(45), "Second")

    def test_fail_grade(self):
        self.assertEqual(get_grade(30), "Fail")

if __name__ == "__main__":
    unittest.main()
```

RED evidence	**Actual result**
Tests	9
Failed tests	3 failures + 1 error
Failing test names	test_average_of_three_marks test_average_of_one_mark test_passing_at_mark test_distinction_grade
Bug mapping	test_average_of_three_marks → Arithmetic bug test_average_of_one_mark → Arithmetic bug test_passing_at_mark → Boundary bug test_distinction_grade → Unreachable branch bug



7. Bug-to-Test Mapping

Bug	Test exposing it	Evidence
Arithmetic	average_of_three_marks / average_of_one_mark	Wrong result / division by zero
Boundary	passing_at_mark	40 returns False
Unreachable branch	distinction_grade	95 returns First

8. RED Commit

git add session8/test_student_utils.py

git commit -m "test: add red student utility tests"

git push

9. Activity C — Fix and Go GREEN

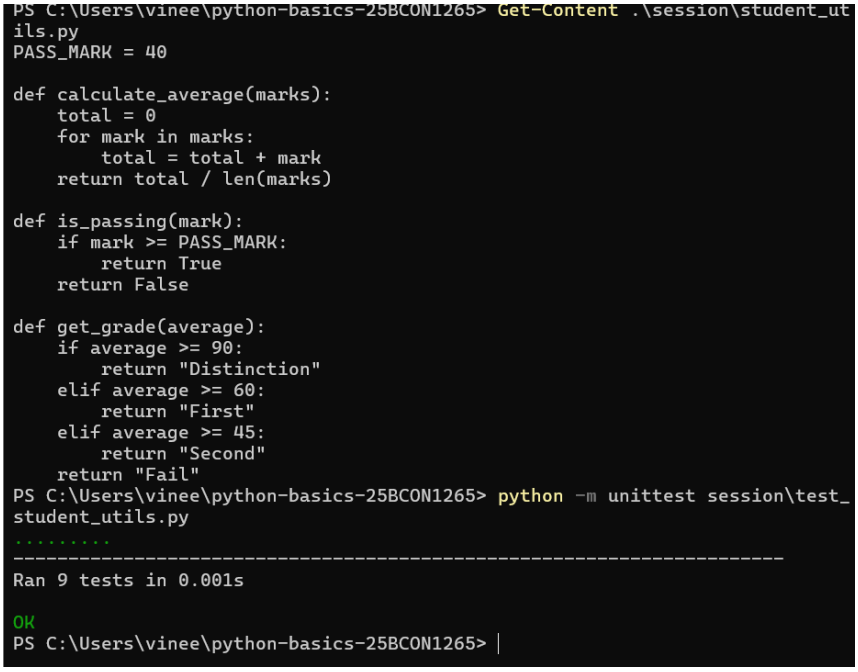
```
PASS_MARK = 40

def calculate_average(marks):
    total = 0
    for mark in marks:
        total = total + mark
    return total / len(marks)

def is_passing(mark):
    if mark >= PASS_MARK:
        return True
    return False

def get_grade(average):
    if average >= 90:
        return "Distinction"
    elif average >= 60:
        return "First"
    elif average >= 45:
        return "Second"
    return "Fail"
```

GREEN evidence	Actual result
Tests run	9
Passed	9
Failed	0
Test file changed?	No



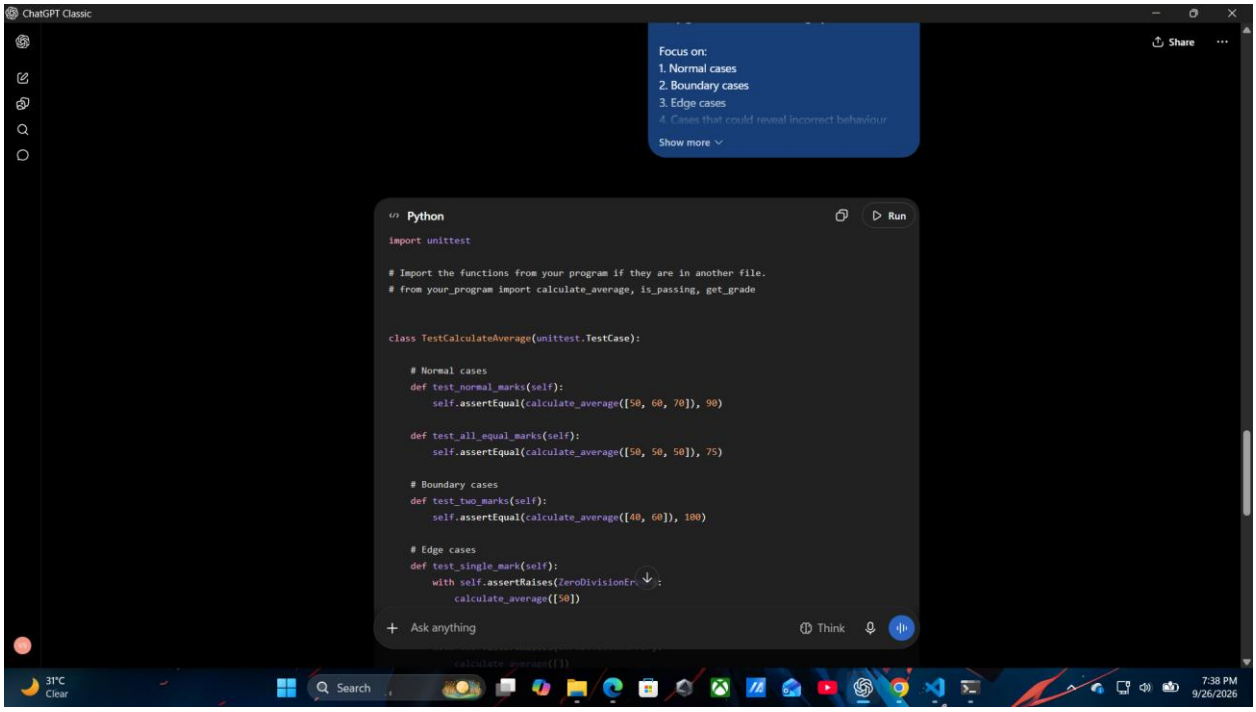
10. AI-Generated Tests from Broken Code

Use a copy of the BROKEN implementation for this experiment. Ask AI to generate tests from the implementation, without asking it to fix the code.

Prompt:

Generate unit tests for this Python module based only on its current implementation and behaviour. Do not correct the implementation. Provide the tests and briefly state what each test expects.

Aspect	Specification suite	Broken-code suite
Source	Required behaviour	Current implementation
Independence	Independent of code	Depends on code
Main purpose	Detect wrong behaviour	May mirror current behaviour
Risk	Can expose bugs	May certify bugs



11. AI Test Experiment — Actual Result

Question	Actual result
Caught arithmetic bug?	NO
Caught boundary bug?	NO
Caught unreachable branch bug?	YES
Pass/fail count on broken code	Not executed
Main difference	AI-generated tests included more cases, but some expected results matched the buggy implementation instead of the specification.

12. Test Suite Comparison

Point	Specification-based	Implementation-based
Basis	Required behaviour	Current code
Independent?	Yes	No
Bug detection	Designed to expose wrong behaviour	May reproduce bugs
Role	Primary RED/GREEN evidence	Comparison experiment

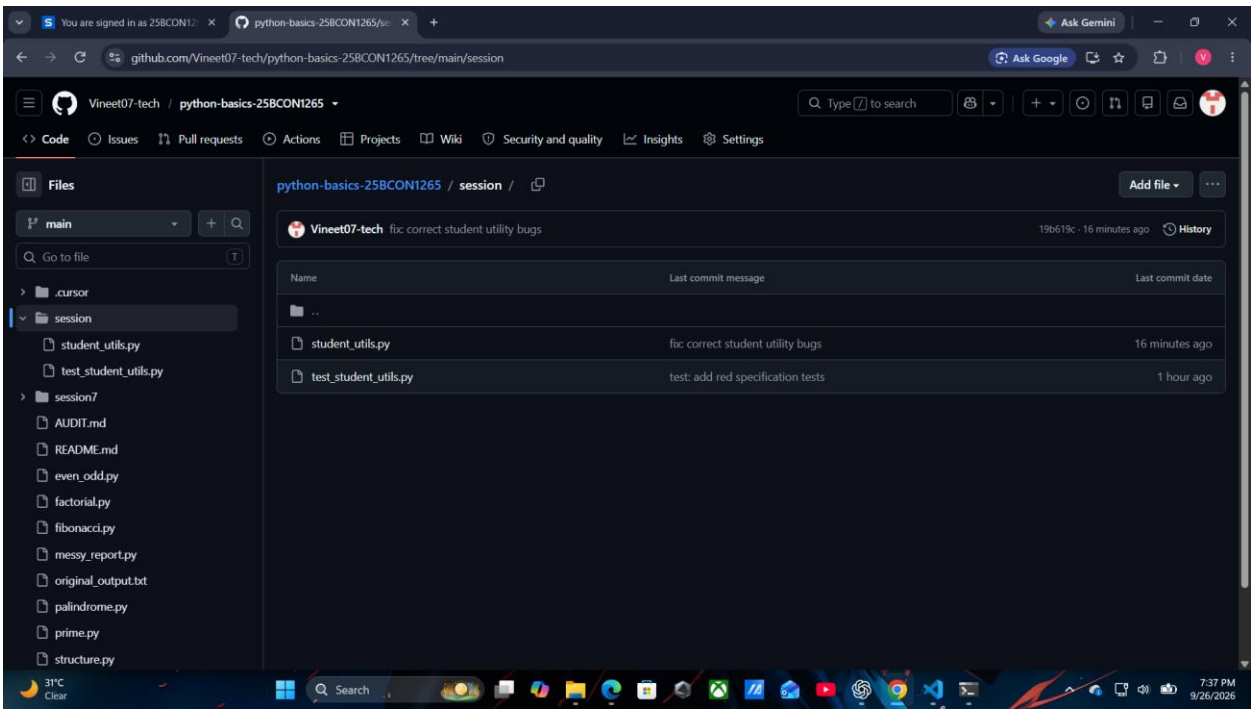
13. README Testing Section

Add this section to README.md, then audit every claim:

Testing
- Test file: `session8/test_student_utils.py`
- Run command: `python -m unittest session8/test_student_utils.py`
- Specification tests: 9
- Student-designed test: 1
- Tests are written from required behaviour.
- RED evidence is committed before the GREEN fix.

14. README Audit

Claim	Verified?
Test file path	YES
Run command works	YES
Test count correct	YES
RED precedes GREEN	YES
No unperformed claim	YES



15. Git Commit Order

Order	Evidence
1	RED test file committed before the fix.
2	Failing test output captured.
3	Fixed module committed in a later GREEN commit.
4	README Testing section updated and audited.

16. Suggested Commit Messages

Stage	Commit
RED	test: add red student utility tests
GREEN	fix: correct student utility bugs
README	docs: update testing documentation

17. Personal Reflection

I learned that fixing a bug is not enough; I need to prove the failure before the fix and the pass afterward. Specification-based tests helped me check required behaviour independently of the implementation. I also learned that AI-generated tests can be misleading when they are based on buggy code, so I should verify tests against the specification.

18. Academic Integrity

I will report the actual RED and GREEN results and preserve the real commit order. I will not reconstruct a failing test after the fix, edit tests just to make them pass, or invent experiment results. Unexpected outcomes will be recorded honestly.